

키보드 마스터

START



최승우



GAME 설명 및 규칙



★ **게임 컨셉:** 위에서 아래로 떨어지는 노트를 타이밍에 맞춰 키보드로 입력하여 점수를 획득하는 정통 리듬 액션 게임

조작 방식: 키보드의 S, D, J, K 4키를 사용

난이도 선택: 게임을 실행해 게임 시작 버튼을 누르면 난이도를 선택 할 수 있고 난이도에 따라 노트가 떨어지는 속도가 다름

SCORE: 0



TIME: 60

READY?

RHYTHM GAME

게임 시작

S

D

J

K

SCORE: 0



TIME: 60

난이도를 선택하세요

READY?

쉬움 (Easy)

보통 (Normal)

어려움 (Hard)

EXTREME (Hell)

S

D

J

K

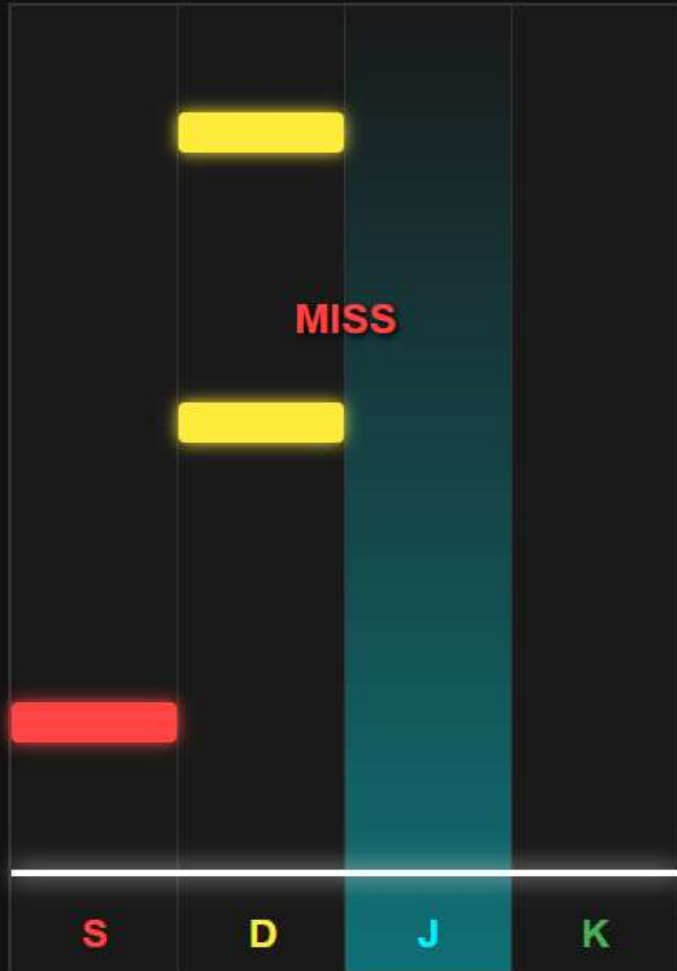


GAME 설명 및 규칙

SCORE: 50



TIME: 57



게임 기본 규칙 (Rules)

[판정 등급 및 보상]

판정	오차 범위	점수	효과
PERFECT	0~39px	100	-
GREAT	40~69px	50	-
GOOD	70~99px	20	-
MISS	범위 이탈	0	목숨-1 / 콤보초기화

타이밍 맞추기

노트 중심이 흰색 판정선에 겹칠 때 키 입력

연속 콤보

연속 성공 시 보너스 점수 및 콤보 텍스트 활성화

생존 모드

목숨 3개 소진 시 게임 즉시 종료 (Survival)

🕒 정확한 타이밍으로 60초간 생존하여 고득점을 달성하세요!

기본 규칙:

노트가 하단의 판정선에 겹치는 순간 해당 키를 누른다.

타이밍의 정확도에 따라 **PERFECT, GREAT, GOOD, MISS** 4단계 판정이 내려짐.

MISS 발생 시 체력이 깎이고 콤보가 초기화됨

PERFECT = 100점

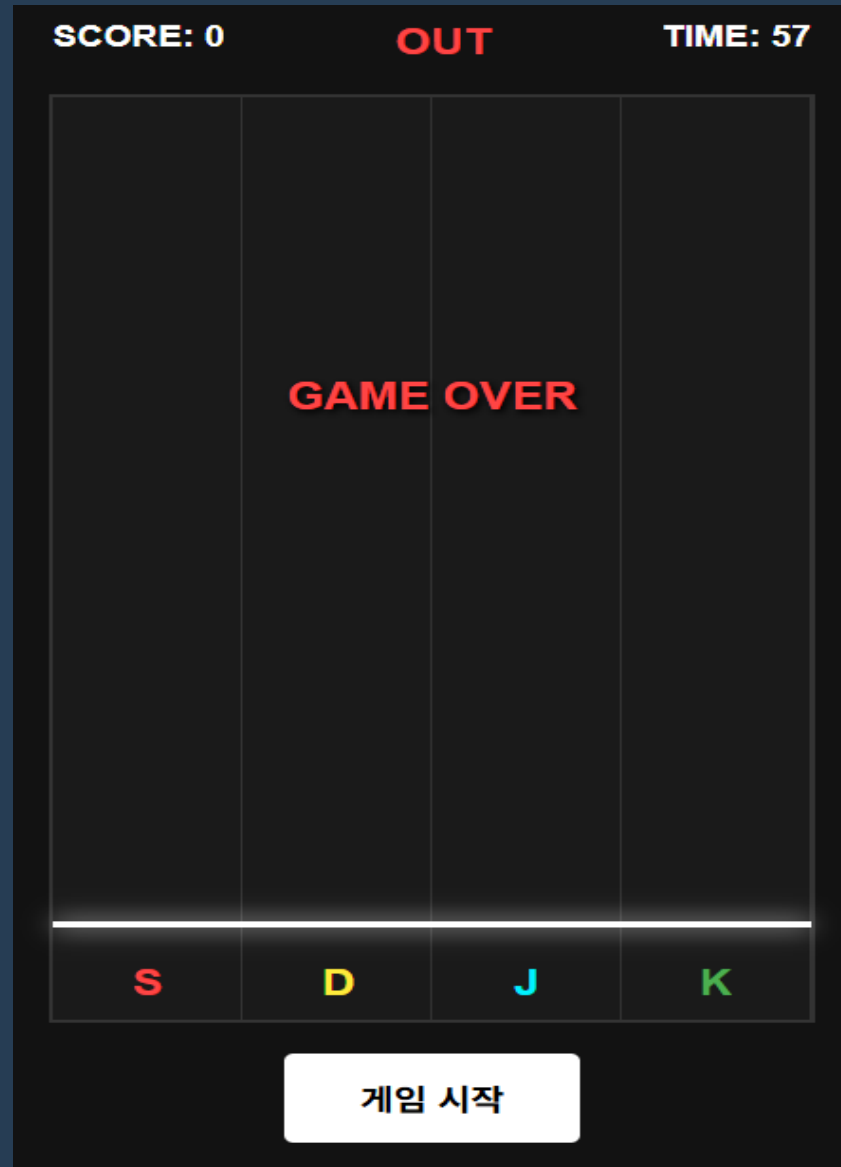
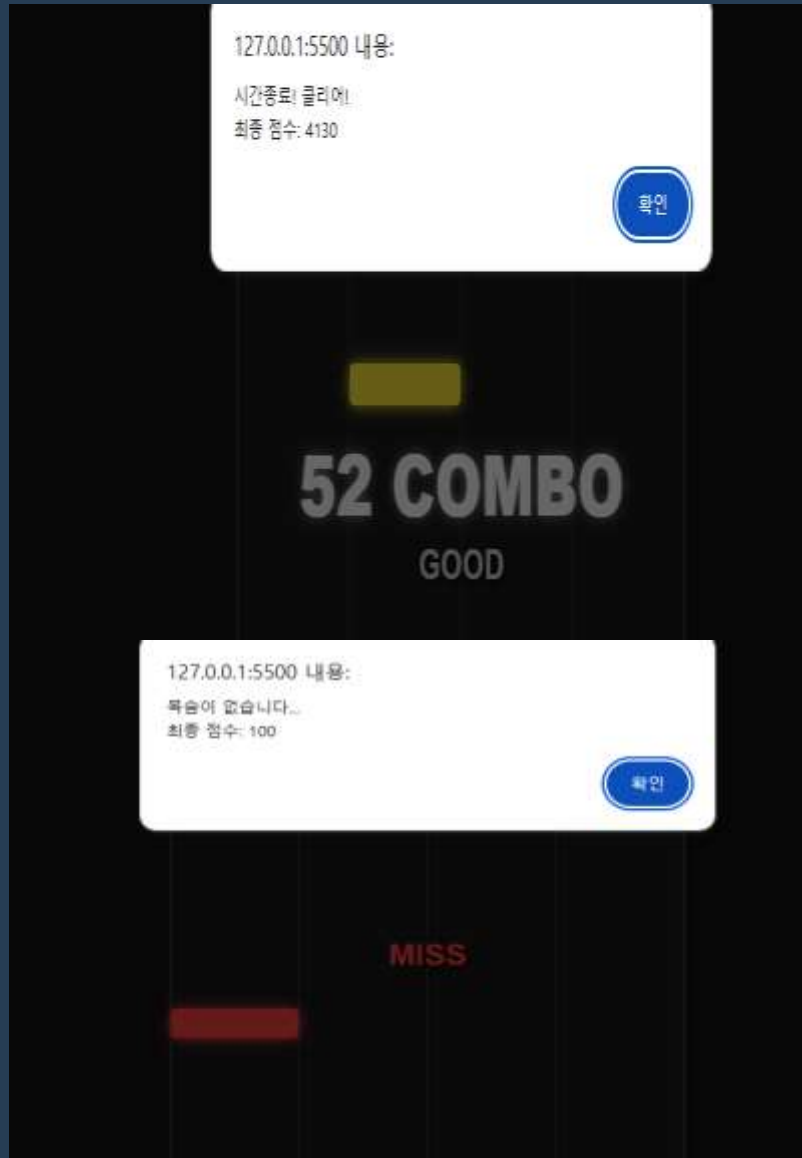
GREAT = 50점

GOOD = 20점

M I S S = 0 점



GAME 설명 및 규칙



기본 규칙:
클리어 시 최종점수가 표시됨
반대로 게임을 클리어 하기 전 목숨을 모두 잃어버리면 획득한 점수가 표시되고 게임이 종료됨
게임 시작을 누르면 난이도 선택과 함께 재시작 가능



GAME

구상

★ 🎱 1단계

시스템 설계 및 아키텍처

컴포넌트 분리: HUD(점수/시간), Game Board(레인/노트), Result Window로 영역 정의.

데이터 구조 설계: * 노트의 정보를 객체(Object)화하여 관리.
S, D, J, K 각 레인을 배열(Array) 인덱스로 매핑

객체 지향적 접근: 각 노드가 스스로 하강 로직과 제거 로직을 가지도록 설계.

★ 🎱 2단계

핵심 메커니즘 구현

노트 엔진 구축: set Interval기반의 비동기 프로세스를 통한 노트 생성 및 프레임 단위 하강 애니메이션 구현

입력 이벤트 처리: 키보드 key down event listener를 통한 실시간 입력 감지.

좌표 기반 판정 로직: 판정선 좌표와 노트 좌표 간의 절대 거리 (Math.abs)를 계산하여 Perfect/Great/Good 판정 알고리즘 완성.

게임 상태(관리: 콤보, 점수, 체력 데이터의 실시간 상태 동기화 시스템 구축

★ 🎱 3단계

게임 시스템 및 UI 최적화

서바이벌 모드 도입: MISS 시 HP 감소 및 HP 0일 때의 예외 처리(Game Over) 추가.

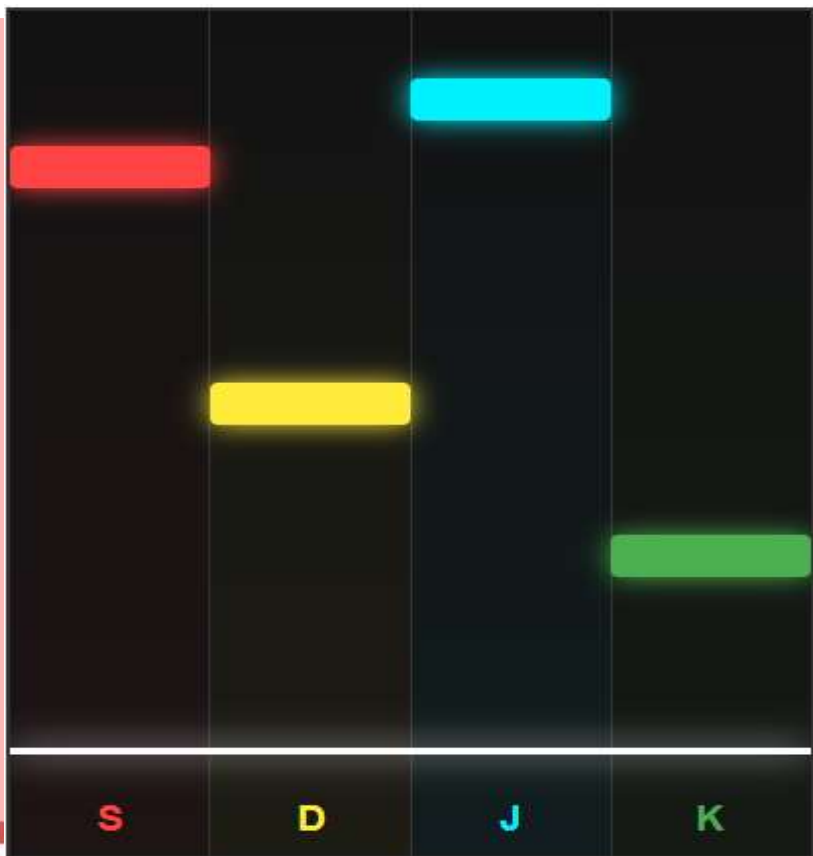
시각적 피드백 강화: * 레인별 네온 컬러 적용 및 키 눌림 시 그라데이션 효과.
판정 결과에 따른 텍스트 컬러 애니메이션.

동적 밸런싱(Dynamic Balancing): 판정 임계치(100px) 설정 및 난이도 별 가변 노트 속도(Variable Note Speed) 최적화
난이도 선택 추가:
(난이도 별 속도가 다름) 쉬움 모드
어려움 모드, **익스트림 모드**



GAME 코드

```
// 0.6초마다 0~3 사이의 난수를 생성하여 레인 결정  
spawnInterval = setInterval(() => {  
  const randIndex = Math.floor(Math.random() * 4); // 결정된 인덱스의 레인에 노트 생성 함수 호출  
  createNote(randIndex);  
}, 600);
```



멀티 레인 동적 노트 생성

구현 원리: Math.random()을 활용
해 0~3 사이의 인덱스를 무작위로
추출하고, 이를 4개의 레인 객체와
매핑하여 노트를 생성.

기술적 포인트: setInterval을 활용
한 **비동기 루프** 구조. **비동기 루프**
구조를 통해 메인 스레드의 간섭 없
이 일정 주기마다 신규 노트 객체를
인스턴스화(Instantiation)

기술 설명: Math.random()과
Math.floor()를 조합하여 정수형 난
수를 생성, 4개의 레인 객체 배열에
인덱스로 접근.



GAME 코드

```
// --- 노트 생성 및 이동 로직 ---
function createNote(idx) {
  const note = document.createElement('div');
  note.className = 'note';
  note.style.top = '-30px'; // 화면 위쪽 안보이는 곳에서 시작

  // 레인 색상 및 네온 효과 부여
  note.style.backgroundColor = laneColors[idx];
  note.style.boxShadow = `0 0 10px ${laneColors[idx]}`;

  laneElements[idx].appendChild(note); // 해당 레인에 노트 추가

  let noteTop = -30;
  // 개별 노트마다 고유의 타이머(setInterval) 할당
  const move = setInterval(() => {
    if (!gameActive) { clearInterval(move); note.remove(); return; }

    noteTop += noteSpeed; // 설정된 속도만큼 y좌표(아래) 이동
    note.style.top = noteTop + 'px';

    // 화면 밖(600px)으로 나갔을 때 리소스 해제(miss 처리)
    if (noteTop > 600) {
      clearInterval(move); // 타이머 정지
      if (note.parentNode) { // 메모리에서 객체 제거
        note.remove(); // 생명력 감소 함수 호출
        handleMiss(); // 미스 처리 함수 호출
      }
    }
  }, 20); // 50FPS 성능 유지
  note.dataset.timerId = move; // 인터벌 ID 저장
}
```

독립적 객체 생명주기 관리

구현 원리: 생성된 각 노드는 고유의 인터벌(Interval) ID를 가짐. 다른 노트와 상관없이 스스로 하강하고, 스스로 소멸.

기술적포인트: 화면 밖으로 나간 노트는 clearInterval로 동작을 멈추고 remove()로 DOM에서 완전히 삭제.

각 노드가 생성 - >
독립적인 **인터벌 ID**를 부여 - >
여러 개 노트가 화면에 나와도 서로
의 이동이나 소멸에 간섭 X





GAME 코드

```
const currentY = parseInt(target.style.top); // 현재 노트 위치  
const distance = Math.abs(currentY - judgeLineY); // 판정선과의 오차 계산
```

```
if (distance < 100) { // 100px 범위 내 성공 인정  
  clearInterval(target.dataset.timerId); // 판정 성공 시 이동 정지  
  target.remove(); // 노트 제거  
  
  // 거리에 따른 차등 등급 부여  
  if (distance < 40) applyJudgment('PERFECT', '#ffffff');  
  else if (distance < 70) applyJudgment('GREAT', '#ffff00');  
  else applyJudgment('GOOD', '#cccccc');  
}
```



거리 기반 정밀 판정 로직

구현 원리: 사용자가 키를 누른 '그 순간'의 노트 Y좌표와 고정된 판정선 Y좌표(540px)의 차이를 절댓값(Math.abs)으로 계산

기술적 포인트: 오차 범위를 40px, 70px, 100px 단계로 설정하여, 단순 성공/실패가 아닌 숙련도에 따른 차등 점수 부여가 가능

기술 설명: 사용자가 키를 누른 **절대 시점**의 좌표를 캡처하여 고정된 판정선 좌표(540px)와의 거리를 절댓값으로 연산





GAME 코드

```
// --- 판정 결과 UI 업데이트 함수 ---  
function applyJudgment(text, color, isMiss = false) {  
  const judgeEl = document.getElementById('judgment');  
  const comboEl = document.getElementById('combo');  
  
  if (isMiss) {  
    combo = 0; // 콤보 초기화  
    comboEl.innerText = "";  
  } else {  
    combo++; // 콤보 증가  
    // 판정별 점수 부여  
    score += (text === 'PERFECT' ? 100 : text === 'GREAT' ? 50 : 20);  
    comboEl.innerText = combo > 1 ? `${combo} COMBO` : "";  
    document.getElementById('score').innerText = `SCORE: ${score}`;  
  }  
}
```

SCORE: 12400

41 COMBO
GREAT



SCORE: 12500

42 COMBO
PERFECT

실시간 상태 동기화 및 UI 피드백

구현 원리: 점수, 콤보, 목숨 등의 내부 변수값이 변할 때마다 즉시 HTML의 innerText 속성을 수정하여 화면에 반영합니다

기술적 포인트: 실시간성입니다. 판정 로직과 UI 업데이트 로직을 직접 연결하여 유저가 행동함과 동시에 (0.02초 이내) 화면의 점수와 콤보가 바뀌는 즉각적인 피드백을 제공합니다.

기술 설명: 게임 내부의 데이터 변수 (State)와 사용자의 눈에 보이는 UI 요소(DOM)를 1:1로 동기화하는 로직

GAME 코드

```
// 게임 종료 처리 함수
function endGame(msg) {
    gameActive = false; // 제어 플래그를 꺼서 모든 이동 로직 중단
    // 실행 중인 모든 무한 루프(노트 생성, 타이머) 정지
    clearInterval(spawnInterval); // 노트 생성 중단
    clearInterval(timerInterval); // 타이머 중단
    alert(`${msg}\n최종 점수: ${score}`);
    document.getElementById('start-btn').style.visibility = 'visible'; // 초기 화면 복구
    document.getElementById('judgment').innerText = "GAME OVER";
}
```

시스템 종료 시퀀스 (Shutdown Logic)

STEP 1

```
gameActive = false;
전역 제어 플래그를 OFF하여 모든 조건부 로직 즉시 차단
```

STEP 2

```
clearInterval(spawnInterval);
메모리 내 상주하는 노트 생성 스케줄러(Timer) 강제 정지
```

STEP 3

```
clearInterval(timerInterval);
게임 시간 카운트다운 인터벌 해제 및 시스템 자원 반납
```

STEP 4

```
showResultPopup();
최종 연산된 데이터를 기반으로 결과 화면 UI 렌더링
```

● SYSTEM SHUTDOWN COMPLETED



시스템 안정 종료 및 리소스 해제

구현 원리: gameActive라는 상태 변수(Flag)를 두어 게임을 제어

기술적 포인트: 리소스 해제 시퀀스. 목숨이 0이 되거나 시간이 종료되면 활성화된 모든 인터벌을 순차적으로 정지

기술 설명: gameActive라는 전역 플래그(Flag) 변수를 통해 게임의 생명주기를 제어하며, 비정상적인 백그라운드 연산을 차단



GAME 느낀 점



느낀 점: 자바스크립트의 비동기 처리(setInterval)와 DOM 조작을 실제 게임 로직에 적용해보며 웹 프로그래밍의 동작 원리를 이해할 수 있었다.

아쉬웠던 점: 음악(BGM)과 난이도별 배경을 구현하지 못해 아쉬웠다.

코드의 기술적 한계 :

setInterval의 불확실성: 자바스크립트의 setInterval은 브라우저의 다른 작업(태스크)에 의해 아주 미세한 지연이 발생할 수 있다.

DOM 기반 렌더링의 한계: 많은 수의 노트를 동시에 처리해야 할 경우, HTML 요소를 직접 생성하고 지우는 방식(DOM 조작)은 브라우저에 부하를 줄 수 있다

